ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Institut für Virtuelle Produktion
Institute of Virtual Manufacturing

DOE and Optimization Project

Optimization and Machine Learning (151-0840-00L)

TA contact: Marino Moeckli (mamoeckl@ethz.ch)

1. Problem Overview

In this project, you will design the **back plate of an electric vehicle accumulator** (see Figure 1) containing three connector cutouts (two circular, one elongated). The goal is to **improve crash safety** by optimizing the placement of these cutouts.

Safety is evaluated using a finite element (FE) simulation (see item A.2), where performance is measured by the **deformation energy absorbed under impact conditions**.

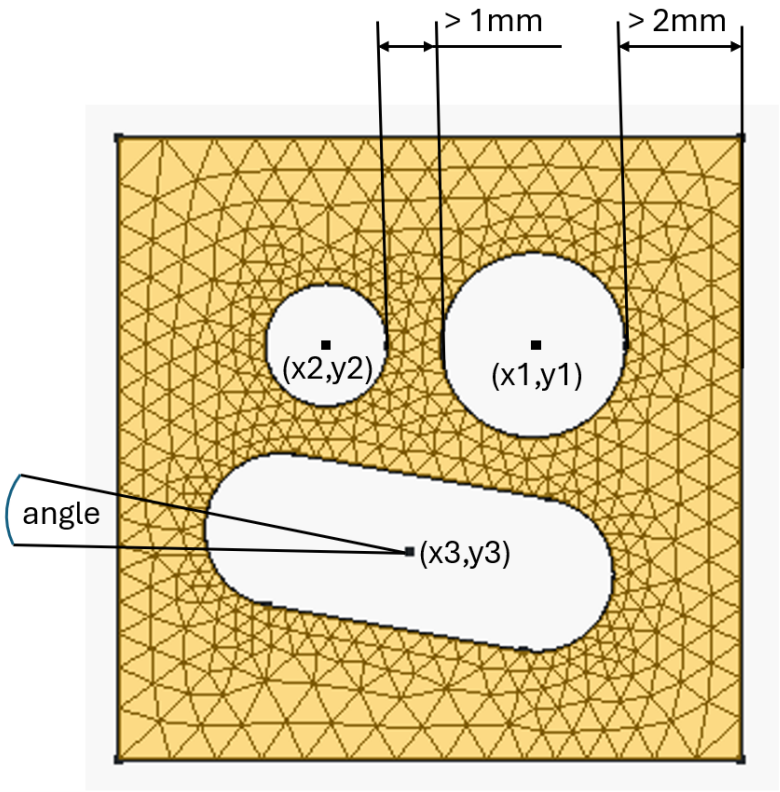


Figure 1: Geometry to optimize with indicated design variables

2. Optimization Problem

Objective

- Maximize **total deformation energy absorbed** at a fixed displacement

Design Variables

Variable	Description
x_1, y_1	Position of circular cutout 1
x_2, y_2	Position of circular cutout 2
x_3, y_3	Position of elongated cutout
angle	Rotation of elongated cutout

Total: 7 design variables

Constraints

Geometry Constraints

Constraint	Value
Plate thickness	Fixed
Cutout shapes	Fixed
Min. distance to edges	≥ 2 mm
Min. distance between cutouts	≥ 1 mm

Material & Mechanical Constraints

Constraint	Value
Material	Aluminum
Max. stress (fracture limit)	275 MPa
Deformation model	Plane strain

3. Your Task

- Optimize cutout placement to **maximize deformation energy**
- Select and implement:
  - Optimization algorithm
  - Sampling / DOE strategy
  - Choose sensible parameter bounds (in the config.json file)
- Extend `main.py` to:
  - Run the optimizer
  - Track objective values
  - Visualize convergence

Evaluation Criteria

Criterion	Description
Efficiency	Number of objective (FEM) evaluations
Convergence	Demonstrated optimizer behavior
Justification	Quality of method explanation

4. Deliverables

- ∼1 page explanation:
  - Optimization strategy
  - Design choices
  - an effort of ∼**8** hours is expected
- 1–3 plots:
  - Convergence behavior
  - Optional comparison of methods

Submission: A .zip folder containing all relevant **code files**, your **config.json file**, and your **report**

5. Setup Instructions

Requirements

- FreeCAD: `https://wiki.freecad.org/Download`
- Python with:
  - numpy
  - scipy
- Project .zip folder from moodle, including:
  - this documentation
  - main.py python file
  - full_journal_fc.py python file
  - main_master.fcstd freecad file
  - config.json file to set parameter bounds

Setup Steps

1. Install FreeCAD (link provided above)
2. Set installation paths of freeCad in:

   `main.py (lines 87)`
3. Activate python environment with numpy and scipy installed
4. Run in console or IDE (i.e vscode):

   `python main.py`
5. Debug, on some machines: If the script does not work initially, copy the full_journal_fc.py file to the bin folder in the FreeCAD 1.0 directory (i.e for windows default 'C:/Program Files/FreeCAD 1.0/bin/full_journal_fc.py') and change the freeCAD_journal variable declared in line 97 of the main.py file to that path. After running like that once, you should be able to copy the file back to the project folder and run normally. Otherwise you can also just leave it like that.
6. You are ready to go: Start developing sampling strategies and optimizers and change the parameter bounds in the config.json file

6. Some Hints

1. To play around with the freecad file, make a copy and then open it and change the parameter values in the spreadsheet and see how the geometry behaves. I would strongly advise not to change the freeCad file or the journal file, except for the installation paths of course.

2. If you decide to use a DOE to build an SVM classifier to learn distinguishing between valid or invalid parameter sets, the designs needed for the DOE do not count to the number of objective evaluations for the optimizer.

3. You are allowed to restructure and use/not use any function or file provided. At the very least, you should change the parameter bounds and extend the main.py function to include an optimizer as well as a plot to visualize the objective value as a function of the number of objective evaluations.

4. Keep your design choice defense brief. As stated earlier, no extensive research and argumentation is required. A good target would be around 1 A4 page of explanation with 1-3 plots showing the performance (i.e comparison of different optimizers, convergence rates, supporting material from the lecture). Remember that a maximum effort of 8h is expected for this project.

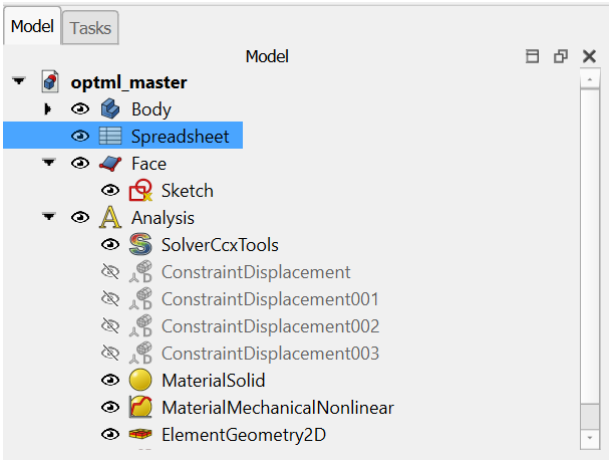
5. Please also submit if the optimizer did not converge, if we see the work and a reasonable explanation of your choice of optimizer, a working solution is not necessarily required for a pass.

Appendix

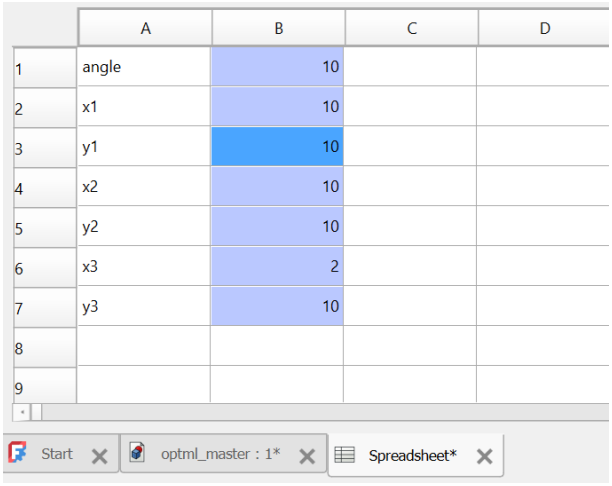
A.1 FreeCad Tutorial - Optional

If you want to play around with the FreeCad model, here is a quick guide on how to change the parameters manually and start the simulation.

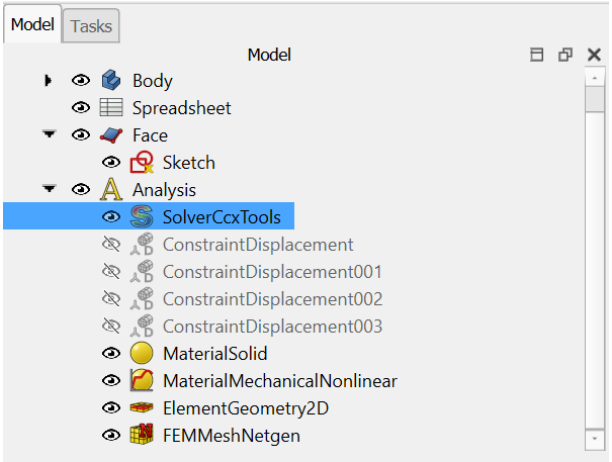
After opening the freecad file (just double click the optml_master file), go to the object tree and click on the spreadsheet.



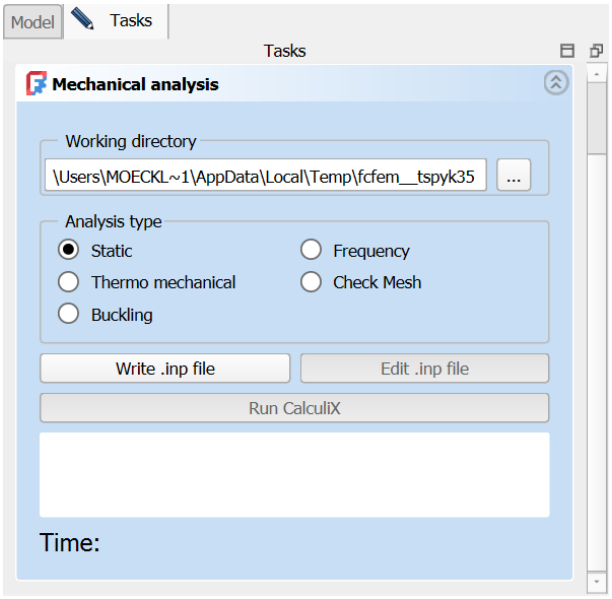
Then, you can change the values in the spreadsheet.



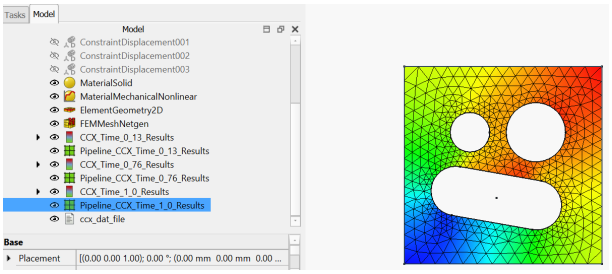
To execute the fem simulation, double click the solver object in the object tree.



Tor run the simulation, click on the *Write .inp file* button, and as soon as the input file is written, you can click the *Run Caclulix* button. When the simulation is finished, you can close this task.



Finally, you can view the results by double clicking the last results object created in the object tree and select the quantity you want to visualize.



A.2 Simulation Setup

In this subsection, we describe the simulation setup and the simplifications introduced to obtain a computationally efficient in-place finite element model (FEM) of the crash scenario.
Ideally, the simulation would model an object impacting the accumulator container plate, causing localized intrusion up to a specified depth. This depth would be limited to ensure that the battery cells remain undamaged. However, accurately representing this scenario requires modeling multiple interacting components and contact conditions, which leads to high computational cost.
To reduce complexity, we instead approximate the problem using a simplified biaxial loading condition applied to the entire plate. This approach represents the stress state at the center of the plate in the more detailed impact scenario, assuming the impact occurs centrally and neglecting geometric discontinuities such as cutouts. While simplified, this approximation captures the essential behavior, as the material undergoes simultaneous stretching in both in-plane directions, similar to the deformation induced by an intruding object.
Rather than defining a maximum intrusion depth, we prescribe a maximum displacement in both axial directions. This displacement serves as an equivalent measure to the intrusion depth in the full impact model.
In the present configuration, the plate lies in the x–y plane. The boundary conditions are defined such that two adjacent edges are constrained in the x and y directions, respectively, while the opposite edges are subjected to the prescribed displacements in their corresponding directions (see Figure 2).

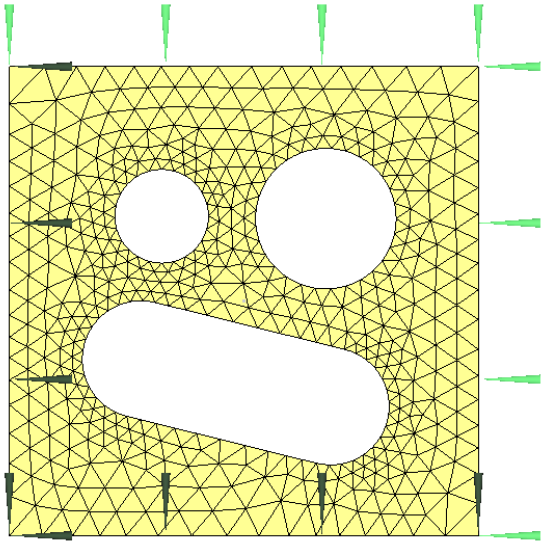


Figure 2: Simplified Biaxial Simulation Setup

A.3 Energy density calculation

If you are interested in how the energy absorption is calculated by the script, here is a short explanation of the simplifying assumptions and exact formulas used to estimate the objective.

The energy absorption is calculated by first estimating the energy absorption density at each node, which is determined in this case as shown in Figure 3 by evaluating the area under the stress-plastic strain curve. The formula used to calculate the area under the curve is given by:

$$E = \sigma_i * \epsilon_{pl} + \frac{\frac{(\sigma_{20} - \sigma_i)}{0.2} * \epsilon_{pl}^2}{2} \tag{1}$$

Where σ_i is the yield stress (onset of plastic deformation), σ_{20} is the stress at 20% plastic strain and ϵ_{pl} is the accumulated plastic strain at a given node. Then, we compute the average density of each triangle and multiply this avarage by the triangle area and the mesh thickness to get an approximated estimate of the energy absorption at each element. Note that for a more precise calculation, we could also estimate the absorbed energy density field given the nodes, the absorbed energy density at the nodes as well as the coordinates and shape functions of the elements, and integrate that density function over the mesh element. In our case, since we are mainly interested in the optimization and sampling, the approximated version is more than enough.

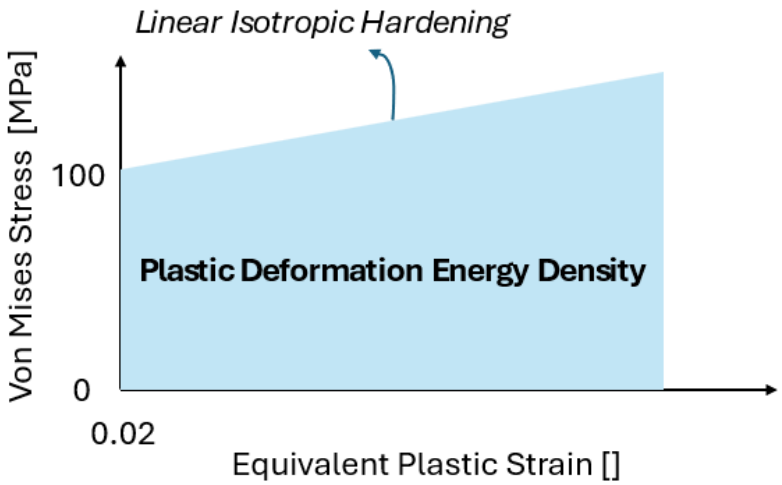


Figure 3: Deformation energy, neglecting the elastic contributions